



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Ingeniería Del Software II

Sección 1

Fundamentos de la Gestión de Proyectos y Productos de Software

Profesor:

Márquez Félix

Alumnos:

Aguilera Rhixeidys V-30.851.503

Bravo Yonkeiner V-30.994.057

Carreño Brayan V-32.015.527

Garcia Nicole V-30.809.865

Ciudad Guayana, septiembre de 2026

Introducción

La ingeniería de software contemporánea enfrenta un panorama de alta complejidad donde el éxito de un sistema tecnológico ya no se mide exclusivamente por la estricta adherencia a un cronograma o a un presupuesto inicial, sino por su capacidad real para generar impacto y resolver problemas críticos del negocio. Históricamente, la disciplina arrastra las secuelas de la denominada "crisis del software", una etapa que evidenció de forma contundente que la mayoría de los fracasos sistémicos no provenían de limitaciones técnicas en la escritura de código, sino de graves deficiencias estructurales en la comunicación, la gobernanza y, sobre todo, en la correcta interpretación de los requerimientos. En respuesta a esta realidad, la dirección de proyectos ha experimentado una profunda reorientación epistemológica, evolucionando desde marcos prescriptivos centrados en entregables rígidos (como el PMBOK 6) hacia esquemas adaptativos fundamentados en principios de valor (PMBOK 7), requiriendo que los profesionales articulen armónicamente las dimensiones de Personas, Proceso, Producto y Tecnología.

El presente informe, estructurado como un análisis crítico de casos prácticos, tiene como objetivo desglosar estos desafíos teóricos y metodológicos mediante la aplicación de estándares internacionales vigentes. En primera instancia, se aborda la elicitación y traducción de necesidades complejas en un entorno hospitalario, empleando normativas como ISO/IEC/IEEE 29148 y SWEBOK V4 para garantizar requerimientos inequívocos y verificables. Posteriormente, se diagnostican los desequilibrios y errores clásicos dentro de ecosistemas de desarrollo desarticulados, donde la imposición de arquitecturas complejas sin respaldo en el talento humano genera cuellos de botella severos, haciendo indispensable la figura del líder servicial. En conjunto, el documento proporciona una hoja de ruta analítica para transitar desde la mera producción operativa de código hacia una ingeniería de productos que garantice la utilidad real de la solución para todos los interesados involucrados.

1. Ejercicio 1. Traducción de Necesidades a Requerimientos y Actores

Contexto

- **Contexto:** Un hospital público necesita reducir el tiempo de espera de los pacientes en urgencias.

1.1 Estándares y Enfoque Aplicado

- **ISO/IEC/IEEE 29148:** Para redactar los requerimientos de forma clara, verificable y sin ambigüedades.
- **SWEBOK V4:** En la parte de elicitación y análisis de requisitos.
- **PMBOK 7ma Edición:** Enfocado en la gestión de interesados y entrega de valor (resolver el problema real de las colas y la atención).

1.2 Actores, Necesidades y Requerimientos

1.2.1 Paciente / Familiar (Usuario final)

- **Necesidad:** Llegan a urgencias y no saben cuánta gente tienen por delante ni cuánto van a tardar, lo que genera angustia y quejas constantes en recepción.
- **Requerimiento (Funcional):** El sistema debe asignar un número de turno al registrarse y mostrar en pantallas de la sala (y mediante consulta por código QR) el estado del turno, el nivel de gravedad asignado y el tiempo estimado de espera en tiempo real.

1.2.2 Enfermero(a) de Triage (Operativo clínico)

- **Necesidad:** El registro manual o en formularios lentos retrasa la clasificación de los pacientes, formando un cuello de botella en la entrada de urgencias.
- **Requerimiento (Funcional):** El sistema debe incluir un módulo de triaje rápido (protocolo estándar de 5 niveles) que sugiera automáticamente la prioridad según los

signos vitales y síntomas principales, completando el registro en menos de 60 segundos por paciente.

1.2.3 Médico de Guardia (Asistencial)

- **Necesidad:** Dificultad para saber a quién atender primero según la gravedad clínica y no solo por el orden de llegada.
- **Requerimiento (Funcional):** El sistema debe presentar una lista de espera ordenada dinámicamente por nivel de gravedad de triaje y tiempo acumulado, permitiendo abrir la ficha clínica resumida con un solo clic al llamar al paciente.

1.2.4 Jefe de Urgencias / Coordinador (Gestión)

- **Necesidad:** No tiene forma de ver en tiempo real si el servicio está saturado para reasignar médicos o abrir más boxes de consulta durante la guardia.
- **Requerimiento (No Funcional):** El sistema debe contar con un dashboard en tiempo real (actualización menor a 10 segundos) que muestre el tiempo promedio de espera, pacientes en cola y active alertas visuales si se superan los tiempos límites permitidos por categoría de triaje.

1.3 Reflexión: Validación mediante Prototipado e IA antes de programar

- **Prototipado rápido (Figma):** La Curva de Boehm demuestra que arreglar un error de requisitos en producción cuesta hasta 100 veces más que en la etapa inicial. Al hacer prototipos navegables podemos probar las pantallas directamente con enfermeros y médicos reales. Si el formulario de triaje resulta incómodo o toma demasiados clics, se corrige en minutos sin haber gastado tiempo programando.

- **Inteligencia Artificial:** Sirve para auditar la redacción de los requisitos bajo el estándar ISO 29148 (detectando términos vagos o no medibles) y para simular escenarios de prueba (por ejemplo, simular qué pasa si llegan 25 pacientes al mismo tiempo por un accidente) para validar las reglas de priorización antes de escribir código.

2. Ejercicio 2. Análisis de las 4 Dimensiones y Errores Clásicos Contexto

2.1 Análisis de los Desbalances en las 4 Dimensiones de Pressman

Según Pressman (2014), el software es un ecosistema definido por cuatro dimensiones fundamentales Personas, Proceso, Producto y Tecnología, que deben mantenerse alineadas para evitar el fracaso del proyecto. En el escenario planteado, se observa un desbalance severo en la interacción de estas dimensiones:

- **Tecnología vs. Personas:** Se presenta una incompatibilidad directa entre la infraestructura elegida y las competencias del equipo. El gerente técnico impone una arquitectura distribuida de microservicios sobre Kubernetes (Tecnología), mientras que el equipo de desarrollo únicamente domina la programación en PHP monolítico (Personas). Forzar una tecnología compleja sin la madurez técnica necesaria paraliza la capacidad de ejecución y genera una brecha insostenible en el talento humano.
- **Proceso (Ausencia de Gobernanza):** La falta de un proceso de gestión de cambios (Proceso) deja al proyecto sin un marco de trabajo que articule el esfuerzo de las personas con la tecnología. Al no contar con un mecanismo de evaluación de impactos, el desarrollo opera de forma caótica frente a las variaciones del entorno.
- **Producto/Alcance (Descontrol de Expectativas):** La dimensión de Producto se distorsiona por la incorporación intempestiva de la función de "pagos con criptomonedas" a mitad del ciclo de vida sin un ajuste presupuestario. Esto destruye la

línea base del proyecto y desvía la entrega de valor del núcleo operativo de la app de delivery.

- **Diagnóstico Integral:** Existe una sobredimensión de la Tecnología impulsada por decisiones unilaterales, un déficit de capacidades en Personas, la omisión total del Proceso y la degradación del Producto debido al descontrol del alcance.

2.2 Identificación de Errores Clásicos

De acuerdo con la literatura de ingeniería de software, el CHAOS Report (Standish Group, 2020) y los principios de Brooks (1975), en este caso se cometen los siguientes errores clásicos:

- **Scope Creep (Alcance No Controlado):** Ocurre al añadir continuamente nuevos requerimientos como la integración de pagos con criptomonedas sin ajustar el presupuesto, el tiempo o los recursos. Este error desestabiliza el proyecto al alterar los compromisos iniciales sin evaluar el impacto real en la entrega.
- **Síndrome de la "Bala de Plata" y Sobre ingeniería Tecnológica:** El gerente técnico asume que adoptar arquitecturas modernas (microservicios en Kubernetes) resolverá automáticamente los retos del sistema, ignorando la Ley de Hofstadter y la realidad operativa del equipo. Esto introduce una complejidad innecesaria que ralentiza el avance y satura al personal.
- **Aceptación de Cambios sin Evaluación de Impacto (Falta de Gobernanza):** Permitir solicitudes del cliente a mitad del desarrollo sin contar con un proceso formal de control de cambios genera un optimismo injustificado. Ignorar el costo del cambio en fases avanzadas deviene en retrabajo, sobrecostos y deuda técnica.

2.3 Propuesta de Mitigación Inmediata desde el Rol de Líder Servicial

Bajo los principios del PMBOK 7 y la filosofía del Líder Servicial (Greenleaf, 1977; Schwaber & Sutherland, 2020), un Gerente de Proyectos moderno debe aplicar las siguientes acciones correctivas:

- **Rediseño Arquitectónico Enfocado en la Simplicidad:** Detener la implementación forzada de microservicios en Kubernetes y reorientar el desarrollo hacia un monolito modular en PHP. Esta decisión alinea la dimensión de Tecnología con las capacidades actuales de las Personas, eliminando la curva de aprendizaje crítica y acelerando la entrega del Producto Mínimo Viable (PMV).
- **Implementación de Gobernanza y Negociación de Alcance:** Establecer de inmediato un marco de trabajo ágil (Proceso) que incorpore una rutina formal de gestión de cambios. El Líder Servicial debe reunirse con el cliente para transparentar el impacto de la función de criptomonedas, aplicando un intercambio de alcance, si se requiere integrar dicha función en la primera entrega, se deberá aumentar el presupuesto o posponer funcionalidades secundarias del *backlog*.
- **Facilitación y Remoción de Impedimentos:** Asumir un rol mediador para proteger al equipo de desarrollo de presiones externas desmedidas y tomar decisiones alineadas con la seguridad psicológica. Paralelamente, se debe trazar un plan de capacitación progresiva para que el equipo adquiera competencias en arquitecturas distribuidas de cara a futuras versiones del producto.

3. Ejercicio 3. Evolución de PMBOK 6 a PMBOK 7 y su Impacto en los Requisitos

3.1 El Cambio de Paradigma: De la Prescripción a los Principios

La evolución entre las ediciones del PMBOK marca una reorientación epistemológica fundamental en la dirección de proyectos.

- **Enfoque Predictivo (PMBOK 6):** Mantenía una estructura prescriptiva fundamentada en áreas de conocimiento y grupos de procesos, donde la meta principal era la producción de entregables (outputs). El éxito de un proyecto se cuantificaba mediante la estricta adherencia al "Triángulo de Hierro" (alcance, costo y tiempo). Este esquema imponía un avance determinista que mostraba severas rigideces frente a la volatilidad de los requerimientos.
- **Enfoque Adaptativo (PMBOK 7):** Abandona la rigidez procesal para adoptar un marco basado en 12 Principios de la Dirección de Proyectos y 8 Dominios de Desempeño. Este estándar prioriza la entrega continua de valor y concibe el proyecto como un subsistema inserto en un ecosistema mayor de la organización.

3.2 Impacto en la Gestión de Requisitos

Este salto cualitativo transforma por completo cómo se entienden y gestionan los requisitos del software.

- **De Outputs a Outcomes:** La gestión migra de la simple generación de entregables técnicos (outputs) a la optimización del impacto y los resultados de negocio (outcomes).
- **Flexibilidad Estratégica:** En lugar de asumir que cumplir con un plan prefijado garantiza la utilidad de la solución, se prioriza el descubrimiento continuo (product discovery).
- **Principios Habilitadores:** Principios del PMBOK 7 como el Pensamiento Sistémico, la Adaptabilidad y Resiliencia, y la Adaptación metodológica (tailoring) permiten gestionar requisitos inestables como un estado natural del negocio, prefiriendo la flexibilidad táctica a la adhesión ciega a una planificación inicial.

3.3 Ejemplos Prácticos de este Cambio

- **Ejemplo 1 (Adaptabilidad frente a especificaciones rígidas):** En un modelo tradicional, los requisitos de un sistema se congelan en la fase de especificación; si el mercado cambia a mitad del desarrollo, el producto final nace obsoleto. Bajo el PMBOK 7, el equipo utiliza bucles rápidos de retroalimentación (feedback loops) para ajustar las características del software según las necesidades emergentes del entorno, aplicando el principio de *Adaptación*.
- **Ejemplo 2 (Medición del Valor sobre el Código Funcional):** Un gerente de proyectos tradicional celebraría el despliegue de 10 nuevos módulos de software en la fecha acordada. Un líder moderno bajo el PMBOK 7 (basado en el principio de *Valor*) evalúa si esos módulos realmente resolvieron el problema del usuario, utilizando métricas reales de adopción, retención y retorno de inversión (ROI) para evitar el "síndrome de la fábrica de funcionalidades".

4. Ejercicio 4. El Verdadero Riesgo en la Ingeniería de Software Moderna

La premisa de que el mayor riesgo es construir un software perfecto que nadie utilice refleja la transición histórica y técnica de la disciplina.

4.1 La Crisis del Software y la Complejidad Esencial

El concepto de "Crisis del Software", acuñado en la conferencia de la OTAN en 1968, evidenció que los proyectos no fracasaban por falta de capacidad técnica para compilar o escribir código, sino por deficiencias estructurales en la comunicación, el liderazgo y la gestión de requisitos.

En la actualidad, esta crisis ha evolucionado hacia la "Complejidad Esencial", donde los proyectos operan en dominios complejos y caóticos (según el modelo Cynefin), haciendo indispensables los enfoques empíricos por encima de los predictivos.

4.2 La Importancia Crítica de la Ingeniería de Requisitos

Los requisitos son el puente indispensable entre las necesidades humanas y las soluciones tecnológicas. Un proyecto con un código excelente, pero que no resuelve el problema de negocio, es, por definición, un fracaso.

El riesgo económico se ilustra con la Curva de Boehm, la cual demuestra que corregir un defecto de requisitos durante la fase de mantenimiento cuesta hasta 100 veces más que detectarlo y corregirlo en la fase temprana de elicitación.

El malentendido de estas necesidades no solo genera desperdicio, sino que acumula una grave deuda técnica.

4.3 La Identificación y Comprensión de los Actores (Stakeholders)

Para evitar construir software inútil, es imperativo romper el paradigma de que el usuario final es el único interesado. Los actores abarcan a patrocinadores, desarrolladores, la sociedad civil (impacto ético), e incluso aquellos interesados en que el proyecto no se concrete, los cuales deben ser gestionados por el Project Manager.

El uso de herramientas de modelado modernas, como el desarrollo de Personas y User Journeys, permite al equipo empatizar profundamente con los contextos reales de los stakeholders, asegurando que la solución tecnológica se alinee con sus problemas subyacentes.

4.4 Mitigación a través de Enfoques Ágiles

Para contrarrestar el riesgo de la "no utilización", el Manifiesto Ágil prioriza la colaboración estrecha con el cliente sobre la negociación contractual.

Al integrar continuamente al usuario final en el bucle de desarrollo y entregar software funcionando de forma frecuente, el equipo de ingeniería se asegura de que el esfuerzo técnico maximice su impacto operativo y comercial, satisfaciendo verdaderamente al mercado.

Conclusión

El análisis desarrollado a lo largo de este informe demuestra que la sofisticación técnica y la calidad del código, por sí solas, resultan insuficientes para garantizar la viabilidad de un proyecto de software si la solución no se encuentra anclada a las necesidades genuinas de sus actores. La correcta evaluación de los requerimientos revela que el costo del cambio, evidenciado a través de la Curva de Boehm, puede destruir la utilidad económica de un desarrollo si los defectos de interpretación no se detectan tempranamente mediante validaciones proactivas como el prototipado rápido. Asimismo, el diagnóstico basado en las cuatro dimensiones fundamentales (Personas, Proceso, Producto y Tecnología) deja en evidencia que forzar imposiciones tecnológicas sobre equipos sin la preparación adecuada, omitiendo simultáneamente la gestión de cambios, conduce irremediablemente a desviaciones destructivas como el scope creep y el fracaso operativo. Queda claro que la adopción de esquemas ágiles y el enfoque orientado a principios del PMBOK 7 constituyen imperativos estratégicos insustituibles frente a la volatilidad del entorno.

En definitiva, se reafirma que el mayor riesgo que asume la disciplina en la actualidad reside en la construcción milimétricamente perfecta de herramientas que nadie utiliza. Superar este paradigma exige que los perfiles técnicos evolucionen hacia líderes facilitadores, enfocados en el descubrimiento continuo de valor (product discovery) y en la mitigación de la brecha entre el usuario final y la arquitectura del sistema. Al abandonar el modelo operativo de ser simples "fábricas de funcionalidades" para integrar sistemáticamente a los stakeholders en el ciclo de vida, los equipos de desarrollo logran maximizar el impacto de su esfuerzo técnico. Este documento concluye que la superación definitiva de las falencias históricas del software requiere la aplicación estricta de normativas consolidadas, combinadas con una capacidad de adaptación resiliente que posicione la resolución del problema humano en el centro del diseño tecnológico.

Referencias

- Boehm, B. W. (1981). Software Engineering Economics. Prentice Hall.
- IEEE Computer Society. (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK V4). IEEE.
- ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148: Systems and software engineering — Life cycle processes — Requirements engineering.
- PMI (Project Management Institute). (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) – 7th Edition. PMI.
- Wiegers, K., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.
- Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.
- Standish Group. (2020). CHAOS Report 2020.
- Brooks, F. P. (1975). The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley.